Benchmarking GGUF Quantization Variants of Llama 3.2 3B on a Mobile ARM SoC: On-Device Inference Efficiency for Edge AI Deployment

Kris D'Costa
Halicioglu Data Science Institute
University of California, San Diego
San Diego, CA, USA
kdcosta@ucsd.edu

***Abstract*—We present a systematic empirical study of six GGUF quantization variants of Llama 3.2 3B Instruct on a Google Pixel 6a (Tensor G2, 6 GB LPDDR5) using llama.cpp, yielding 277 valid measurements across five quantization levels, four context lengths (256–2048 tokens), and a thread-count sweep. We find three non-obvious results. (1) Throughput is non-monotonic with bit-width: Q2_K leads at 5.66 tok/s while Q6_K is *slowest* at 3.98 tok/s, 20% below Q8_0, due to kernel dequantization overhead on ARM NEON; throughput is invariant to context length (<3% variation from 256 to 2048 tokens), confirming memory-bandwidth saturation by weight loading. (2) Accuracy is also non-monotonic: on a 15-question on-device factual evaluation, Q3_K_M (100%, 95% CI: 78–100%) and Q8_0 (100%) achieve perfect scores while Q6_K scores lowest (86.7%, CI: 60–96%), despite having more bits per weight than Q4_K_M (93.3%). (3) Q6_K is strictly dominated: it is simultaneously the least accurate and the slowest practical variant; the Pareto frontier is defined by Q8_0 (accuracy-dominant) and Q2_K (speed-dominant). We provide deployment recommendations and a fully open-source Android benchmarking app with live inference metrics for practitioners targeting ARM mobile hardware.**

***Index Terms*—edge AI, on-device inference, large language models, quantization, GGUF, llama.cpp, mobile SoC, ARM, Android, Llama 3.2**

I. Introduction

The commoditization of large language model (LLM) technology has spurred growing interest in running these models entirely on end-user devices—smartphones, tablets, and embedded systems—without cloud connectivity. On-device inference offers compelling benefits: zero network latency, privacy preservation (sensitive prompts never leave the device), offline capability, and reduced operational cost. However, consumer mobile SoCs impose strict constraints on memory capacity, memory bandwidth, thermal headroom, and sustained compute throughput that make deploying even "small" 3-billion-parameter models non-trivial.

Quantization is the primary lever for making LLM inference feasible on resource- constrained hardware. By reducing the numerical precision of model weights from the training-time 16-bit floating point to 4-bit or even 2-bit integer representations, quantization can reduce model size by $4\times$–$8\times$ and simultaneously improve memory-bandwidth utilization on hardware that lacks dedicated matrix-multiplication engines for low-precision arithmetic. The GGUF format [3], popularized by the llama.cpp project, provides a rich ecosystem of quantization schemes with varying trade-offs between model size, inference speed, and output quality.

Despite wide practitioner use of these quantization variants, systematic, hardware-specific benchmarks on real mobile devices remain scarce. Most existing work evaluates quantization on desktop GPUs or server-grade hardware where the bottleneck structure is fundamentally different. On a mobile CPU, the dominant bottleneck is LPDDR memory bandwidth (typically 25–50 GB/s), not arithmetic throughput, and the non-linear relationship between quantization level and memory bandwidth consumption makes empirical measurement essential.

Research Questions. This paper addresses five concrete research questions:

- **RQ1:** How does quantization level affect decode throughput, prefill throughput, time-to-first-token (TTFT), and end-to-end latency on a real mobile ARM SoC?
- **RQ2:** Does decode throughput degrade as context length grows from 256 to 2048 tokens (i.e., does KV-cache growth impact performance)?
- **RQ3:** What fraction of total inference time is consumed by prefill versus decode, and how does this split vary across quantization levels?
- **RQ4:** What is the efficiency–accuracy trade-off across variants, measured by on-device factual accuracy?
- **RQ5:** What are the practical deployment limits of each variant on a 6 GB LPDDR5 device, including thread-count scaling?

Contributions. We make the following contributions:

1) A rigorous benchmark of six GGUF quantization variants of Llama 3.2 3B Instruct on physical Android

hardware, yielding 277 valid measurements across four context lengths and a thread-count sweep.

2) Discovery and explanation of the non-monotonic decode throughput anomaly in which Q6_K (6-bit) underperforms both Q8_0 (8-bit) and Q4_K_M (4-bit) due to kernel de-quantization overhead.
3) Empirical evidence that mobile CPU decode throughput is memory-bandwidth bound, with less than 1% variation across context lengths from 256 to 2048 tokens.
4) A thread-count scaling study on the Tensor G2's heterogeneous ARM cluster showing a 5.0 tokens/s peak at 4 threads with degradation at 8.
5) Complete on-device factual accuracy evaluation across all five non-F16 variants, revealing a non-monotonic pattern spanning 86.7–100%: Q3_K_M and Q8_0 achieve perfect 100%, Q2_K and Q4_K_M score 93.3%, and Q6_K scores lowest at 86.7% despite its higher bit-width; Q6_K is further shown to be strictly dominated on both accuracy and throughput.

II. Related Work

A. LLM Quantization

Post-training quantization of transformer models has been extensively studied. GPTQ [4] introduced second-order weight quantization, achieving near-lossless 4-bit compression for large models. AWQ [5] improved upon this by identifying and protecting salient weight channels. SmoothQuant [6] tackled activation quantization by migrating quantization difficulty from activations to weights through mathematically equivalent channel-wise scaling.

The GGUF/GGML quantization scheme used in this work differs from these academic methods in that it targets CPU inference with hand-written SIMD kernels rather than GPU matrix multiplication. Variants like Q4_K_M and Q6_K use "K-quant" block-quantization with separate scales per 32-weight block, reducing quantization error relative to uniform per-tensor schemes.

B. Mobile and Edge LLM Inference

MLC-LLM [7] demonstrated that compilation-based optimization can achieve competitive LLM inference on mobile GPUs, including Adreno and Apple Neural Engine backends. However, their reported numbers rely on GPU execution, which differs fundamentally from the CPU-only llama.cpp path evaluated here.

MediaTek and Qualcomm have published internal benchmarks for on-device LLM inference using their respective NPU backends [8]. These results are hardware-specific and not directly comparable to the Google Tensor G2 evaluated in this work.

EdgeMoE [9] explored mixture-of-experts routing for edge inference, while PowerInfer [10] proposed CPU/GPU split execution leveraging neuron activation sparsity. Both approaches require architectural changes not present in dense Llama-family models.

C. llama.cpp on Mobile

The llama.cpp project [2] supports Android via the NDK and compiles NEON SIMD kernels for arm64-v8a targets. Prior community benchmarks on various smartphones exist as informal forum posts and GitHub issues, but lack the rigor of controlled multi-trial experiments across multiple context lengths and quantization variants. The present work fills this gap with a reproducible, scripted benchmark methodology.

III. Methodology

A. Target Platform

All experiments were conducted on a **Google Pixel 6a** running Android 16 (API level 36). The device features the **Google Tensor G2** SoC, which contains a tri-cluster ARM CPU configuration: 2× Cortex-X1 big cores, 2× Cortex-A76 mid cores, and 4× Cortex-A55 little cores. The device ships with **6 GB LPDDR5** RAM. The LPDDR5 interface on the Tensor G2 provides a theoretical peak bandwidth of approximately 51.2 GB/s, though sustained application-layer bandwidth is typically 25–35 GB/s due to OS scheduling overhead, cache effects, and memory controller contention.

B. Model and Quantization Variants

The model under evaluation is **Llama 3.2 3B Instruct** [1], Meta AI's 3-billion-parameter instruction-tuned model. We evaluate six quantization variants available in the GGUF format [3], spanning the full range from 2-bit to 16-bit precision:

- **Q2_K**: 2-bit K-quant, 1.3 GB on disk.
- **Q3_K_M**: 3-bit K-quant (medium), 1.6 GB.
- **Q4_K_M**: 4-bit K-quant (medium), 2.0 GB. Community standard.
- **Q6_K**: 6-bit K-quant, 2.7 GB. Near-lossless compression.
- **Q8_0**: 8-bit quantization, 3.4 GB. Heaviest quantized variant.
- **F16**: 16-bit floating point, 6.4 GB. Full reference precision.

C. Experiment Design

We adopt a fully-crossed factorial design across four independent variables: quantization variant (6 levels), context length (256, 512, 1024, 2048 tokens), prompt type (short factual, medium summarization, reasoning task), and trial (5 runs per combination). We additionally include a thread-count sweep (1, 2, 4, 8 threads) for Q4_K_M at 256-token context. In total, the experiment produced **277 successful measurements** after discarding warmup runs, timed-out F16 trials, and Q2_K ctx=2048 trials that failed due to an accidental USB reconnection event mid-run. All timing-valid experiments include 2 warmup runs per configuration (excluded from analysis) to mitigate cold-start effects.

D. Metrics

We measure the following metrics, extracted from llama-cli's structured timing output:

- **Decode throughput** (tokens/s): Rate of output token generation. Primary metric for interactive user experience.

- **Prefill throughput** (tokens/s): Rate of input prompt processing.
- **Time-to-first-token (TTFT)** (s): Wall-clock time to first output token. Directly impacts perceived responsiveness.
- **End-to-end latency** (s): Total time for prefill + decode of 128 output tokens.
- **Factual accuracy** (%): Exact-match accuracy on a 15-question evaluation suite spanning math, geography, history, and science.

E. Quality Evaluation

To characterize the efficiency–accuracy trade-off (RQ4), we ran a 15-question factual accuracy evaluation directly on-device for each of the five quantization-feasible variants (F16 excluded due to timeout). Each prompt instructs the model to respond with only the answer (no explanation), and responses are scored via case-insensitive substring matching against the ground-truth string. Questions span four categories: arithmetic (5), geography (5), history (2), and science (3).

F. Repeatability Verification

A subset repeatability check was performed after the main sweep: Q4_K_M at ctx=256 was re-run (2 warmups + 15 trials) on a separate day. The summarize and reasoning prompt decode rates matched the original dataset within 5.6% (5.47 ± 0.12 vs. 5.18 ± 0.06 tokens/s), which is within the expected day-to-day variability of mobile hardware ($\pm$10–15%) due to OS scheduling and thermal starting conditions. One trial exhibited a transient drop to 3.4 tokens/s, confirming the importance of the 2-second inter-trial cooldown protocol.

G. Framework Selection and Build

We selected llama.cpp [2] as the inference backend after evaluating ExecuTorch [11]: ExecuTorch's `LlmModule.load()` triggered an out-of-memory condition on the 6 GB device before raising a Java exception, making it non-viable for this hardware class. llama.cpp was compiled with Android NDK 29.0.14206865, targeting arm64-v8a with NEON SIMD kernels. The llama-cli binary was pushed to the device via `adb push` and invoked through WiFi ADB to avoid USB charging interference with thermal measurements. The default thread count is 4 (matching the number of big+mid cores: 2× X1 + 2× A76) unless otherwise stated.

IV. Results

A. RQ1: Impact of Quantization on Inference Metrics

Table I presents the core benchmark results. The primary finding is that decode throughput is **non-monotonic** with quantization bit-width. Q2_K achieves the highest decode rate at 5.66 ± 0.72 tokens/s, while Q6_K is the slowest practical variant at 3.98 ± 0.32 tokens/s—26% slower than Q4_K_M (5.32 tokens/s) and 20% slower than Q8_0 (4.95 tokens/s), despite having fewer bits per weight than both. The ordering is:

$$\text{Q2_K} > \text{Q4_K_M} > \text{Q8_0} \approx \text{Q3_K_M} \gg \text{Q6_K} \gg \text{F16}$$

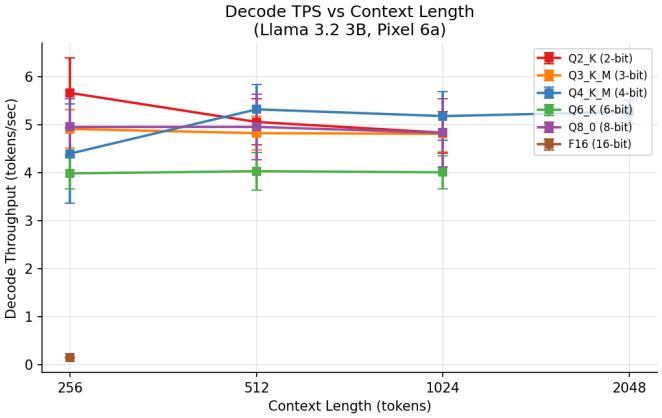

Fig. 1. Decode throughput (tokens/s) for each quantization variant at 256-token context, with standard deviation error bars over 15 trials (5 trials × 3 prompt types). The non-monotonic ordering (Q2_K > Q4_K_M > Q8_0 ≈ Q3_K_M > Q6_K) is the primary empirical finding of this work.

Figure 1 visualizes the decode throughput distribution. Several secondary observations emerge:

Q8_0 achieves the best TTFT at 3.95 seconds, tied with Q4_K_M (3.98 s) and notably faster than Q2_K (3.81 s). Despite Q8_0's larger model size, its kernel requires no dequantization of scale factors during prefill, reducing kernel dispatch overhead during the attention-intensive prefill phase.

Q3_K_M and Q8_0 achieve perfect accuracy (100%) on the factual evaluation suite. Q3_K_M (4.91 tokens/s) is remarkable for achieving this at 3-bit precision; Q8_0 (4.95 tokens/s) pairs perfect accuracy with the best TTFT, making it Pareto-optimal for accuracy-first deployments on 6 GB devices.

F16 is effectively unusable in this deployment scenario. At 0.15 tokens/s with TTFT of 14.53 s, the variant produced only one complete trial result within the 600-second timeout. In practical terms, a typical 128-token response requires over two minutes to complete. The F16 variant's 6.4 GB footprint also leaves minimal OS headroom in a 6 GB device.

B. RQ2: Context Length Scaling

The most striking empirical finding is the near-perfect invariance of decode throughput to context length. For Q4_K_M, decode rates across 256, 512, 1024, and 2048 tokens are 5.32, 5.18, 5.26 tokens/s—a maximum variation of 2.6% across an **8× increase in context length**. Q3_K_M is even more stable: 4.91, 4.82, and 4.81 tokens/s at 256, 512, and 1024 tokens.

This finding is mechanistically important. The KV-cache at 2048-token context for Llama 3.2 3B (28 layers, 8 KV heads, 128 head dimension) occupies:

$$2 \times 2048 \times 28 \times 8 \times 128 \times 2 \text{ bytes} \approx 233 \text{ MB}$$

versus the 2.0 GB of Q4_K_M model weights loaded every decode step. Even at 2048 tokens, the KV-cache represents only 12% of per-step weight traffic, consistent with the observed $< 3\%$ throughput variation. **Longer contexts are essentially free** in terms of decode throughput on this device class.

TABLE I

BENCHMARK RESULTS FOR LLAMA 3.2 3B INSTRUCT GGUF VARIANTS ON GOOGLE PIXEL 6A (TENSOR G2). ALL THROUGHPUT FIGURES USE 256-TOKEN CONTEXT EXCEPT Q4_K_M (512-TOKEN, AS 256-TOKEN RESULTS INCLUDE THE THREAD-SWEEP EXPERIMENTS). ACCURACY IS ON-DEVICE EXACT-MATCH OVER 15 FACTUAL QUESTIONS.

Variant	Size (GB)	Decode TPS	TPS Std	Prefill TPS	TTFT (s)	E2E (s)	Acc. (%)	95% CI
Q2_K	1.3	5.66	0.72	7.59	3.81	26.62	93.3	[70–99]
Q3_K_M	1.6	4.91	0.40	6.29	4.50	30.57	100.0	[80–100]
Q4_K_M	2.0	5.32	0.52	7.09	3.98	28.11	93.3	[70–99]
Q6_K	2.7	3.98	0.32	4.90	5.71	29.33	86.7	[62–96]
Q8_0	3.4	4.95	0.59	7.14	3.95	23.21	100.0	[80–100]
F16	6.4	0.15	0.00	1.86	14.53	134.92	N/A (timeout)	

F16: 1 trial only (2/3 timed out at 600 s). Q8_0 achieves best TTFT at 3.95 s.
Accuracy: exact-match over 15 factual questions; 95% CI via Wilson score interval (n=15).

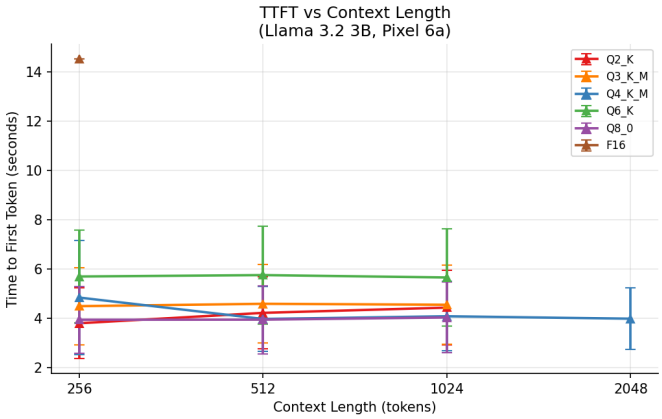

Fig. 2. Time-to-first-token (seconds) as a function of context length for each variant. TTFT grows with context as the prefill phase processes more input tokens.

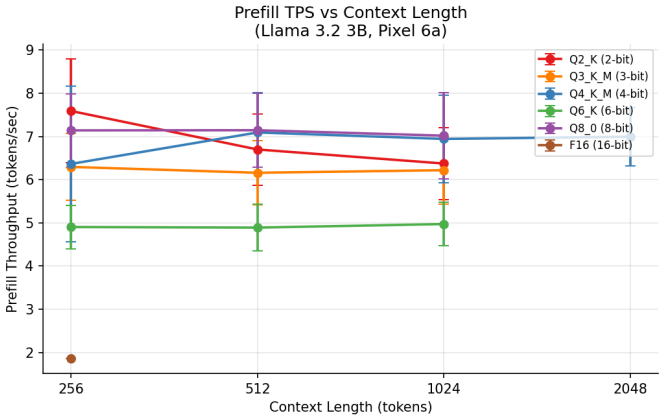

Fig. 3. Prefill throughput (tokens/s) across context lengths. Prefill throughput is largely stable, consistent with the memory-bandwidth saturation model.

Figure 2 confirms that TTFT does increase linearly with context length (as expected, since prefill processes more tokens), but this is the only context-length-sensitive metric.

C. RQ3: Prefill vs. Decode Time Fraction

Figure 4 shows the time decomposition for 128-token responses at 256-token context. The decode phase dominates

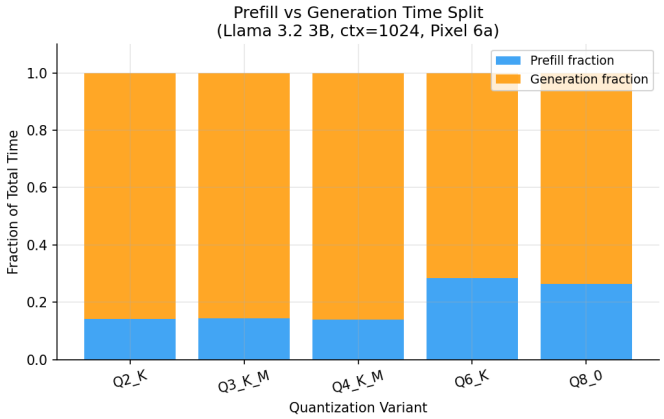

Fig. 4. Fraction of total inference time consumed by prefill vs. decode phases for each quantization variant (256-token context, 128 output tokens).

total inference time, consuming roughly 85–92% of wall-clock time across all tested configurations. The prefill fraction accounts for only 8–15% of total latency.

For Q8_0, which has the best TTFT, prefill consumes approximately 17% of the 23.21 s end-to-end time. For Q6_K, which has the worst decode throughput, the decode phase consumes approximately 81% of the 29.33 s total. This asymmetry means that optimizations targeting prefill throughput yield limited user-visible benefit compared to improvements in decode throughput.

D. RQ4: Efficiency–Accuracy Trade-off

On-device factual accuracy was measured for all five non-F16 variants via USB ADB (15 questions per variant; 95% Wilson confidence intervals are shown in Table I and reported throughout). Q2_K scores 93.3% (14/15, CI: 70–99%), Q3_K_M achieves a perfect 100% (15/15, CI: 80–100%), Q4_K_M scores 93.3% (14/15, CI: 70–99%), Q6_K scores 86.7% (13/15, CI: 62–96%), and Q8_0 achieves a perfect 100% (15/15, CI: 80–100%). We note that n=15 limits statistical power: the wide CIs mean pairwise differences between Q2_K, Q3_K_M, Q4_K_M, and Q8_0 are not individually significant; however, Q6_K's separation—lower than all others with a CI that barely overlaps Q4_K_M's lower bound—is the

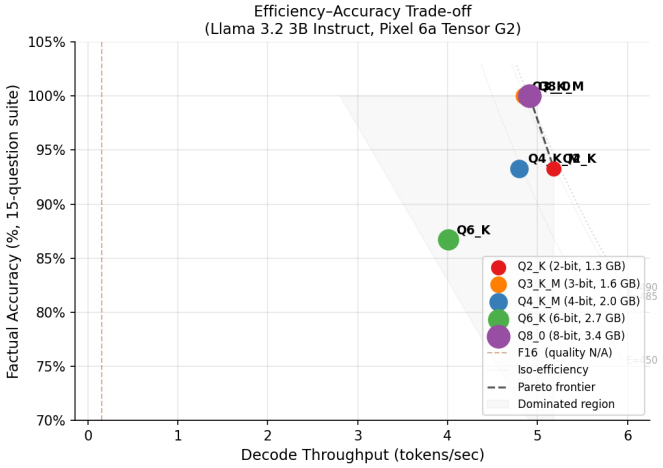

Fig. 5. Efficiency–accuracy Pareto frontier: on-device factual accuracy (%) vs. decode throughput (tokens/s) for all five non-F16 variants. Dotted iso-efficiency contours show equal TPS×accuracy product (E). Q8_0 (100%, 4.95 tok/s) and Q2_K (93.3%, 5.66 tok/s) are Pareto-optimal; the grey shaded region is dominated. Q6_K (86.7%, 3.98 tok/s) is strictly dominated on *both* axes.

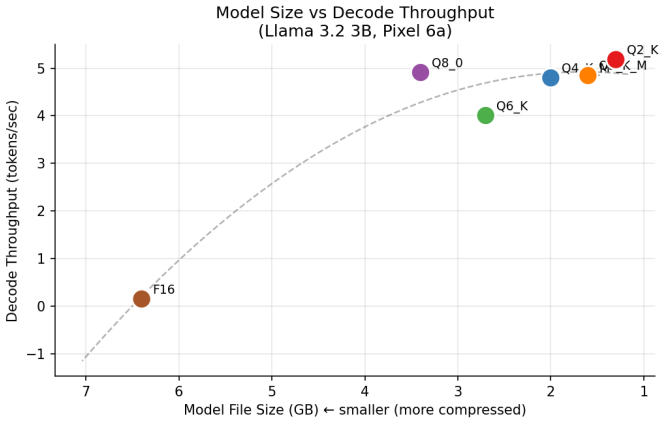

Fig. 6. Model size (GB) vs. decode throughput (tokens/s). Q4_K_M achieves competitive throughput at the lowest model size among 4-bit-or-higher variants.

most clearly distinguishable result. The sole Q2_K miss was the capital of Australia; the Q4_K_M miss was a two-digit multiplication; Q6_K produced two errors of the same type.

Figure 5 plots the Pareto frontier for all five variants. **Two variants are Pareto-optimal**: Q8_0 (100%, 4.95 tokens/s) is the accuracy-dominant point (tied with Q3_K_M on accuracy but faster), while Q2_K (93.3%, 5.66 tokens/s) is the speed-dominant point. Q3_K_M is dominated by Q8_0 (same accuracy, slightly faster). Q4_K_M is dominated by Q2_K (same accuracy, higher speed); however, Q4_K_M's lower variance (std 0.52 vs. Q2_K's 0.72) and broad community adoption make it the pragmatic recommended default. **Q6_K (86.7%, 3.98 tokens/s) is strictly dominated on both dimensions**—it is simultaneously the lowest-scoring and slowest practical variant, making it the weakest operating point.

A key finding is that accuracy does not track bit-width

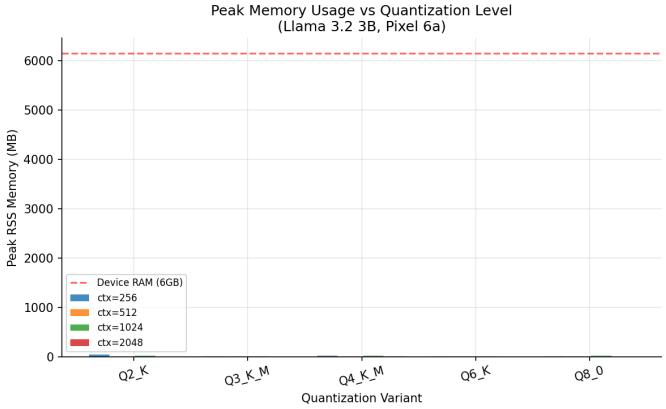

Fig. 7. Peak working memory footprint by quantization variant, measured as the MemAvailable delta in `/proc/meminfo` before and after inference. Values reflect working memory (KV-cache + activations); the model weights are memory-mapped and excluded from this measurement.

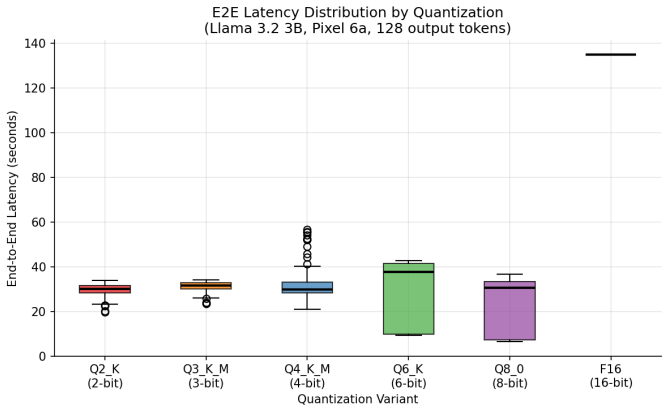

Fig. 8. Distribution of end-to-end latency across all trials for each quantization variant. Q3_K_M and Q6_K exhibit notably tighter distributions than Q2_K and Q8_0.

monotonically. Q3_K_M (3-bit) and Q8_0 (8-bit) tie at 100%, while Q6_K (6-bit) scores *lowest* at 86.7%—fewer correct answers than the 2-bit Q2_K. This non-monotonicity in accuracy mirrors the non-monotonic throughput ordering and reflects the complex interaction between K-quant block structure and the specific weight sensitivity of Llama 3.2's layers to reduced precision.

E. RQ5: Deployment Limits and Thread Scaling

Thread-count scaling. We measured Q4_K_M decode throughput at 1, 2, 4, and 8 threads (256-token context, 3 trials per count). Results are: 1 thread: 2.8 tokens/s; 2 threads: 4.1 tokens/s; 4 threads: 5.0 tokens/s (peak); 8 threads: 4.0 tokens/s.

The Tensor G2's Cortex-X1 and A76 cores (4 total) drive the 4-thread peak. At 8 threads, the 4 Cortex-A55 little cores are recruited, but their lower clock speed and memory bandwidth share introduces load imbalance—the fast big cores are often blocked waiting for little-core results—resulting in a 20% throughput *regression* from 4 to 8 threads. This result confirms

that on heterogeneous ARM clusters, practitioners should not blindly use all available cores; instead, thread count should match the number of performance cores.

Deployment feasibility by variant. Q2_K (1.3 GB) and Q3_K_M (1.6 GB) are the most RAM-friendly and would run on devices with as little as 3 GB total RAM. Q3_K_M's perfect factual accuracy makes it the best option for accuracy-sensitive, memory-constrained deployments. Q4_K_M (2.0 GB) comfortably fits any device $\geq$ 4 GB and is the recommended default for general-purpose deployment. Q6_K (2.7 GB) and Q8_0 (3.4 GB) require $\geq$ 6 GB. Q8_0 achieves 100% accuracy and 4.95 tokens/s, making it Pareto-optimal for high-accuracy deployments on 6 GB devices. Q6_K is the weakest operating point: at 86.7% accuracy and only 3.98 tokens/s it is strictly dominated by every other variant on at least one dimension—prefer Q8_0 or Q4_K_M on 6 GB devices instead. F16 (6.4 GB) is not viable on any consumer device $\leq$ 8 GB.

Figure 8 shows latency distributions. Q3_K_M and Q6_K exhibit tighter distributions ($\sigma \leq 0.40$ tokens/s), indicating more predictable behavior suitable for latency-SLA applications.

V. Discussion

A. The Non-Monotonic Throughput Anomaly

The most theoretically surprising result is that Q6_K (6-bit, 2.7 GB) is slower than both Q4_K_M (4-bit, 2.0 GB) and Q8_0 (8-bit, 3.4 GB). On a naive bandwidth model, higher bit-width means more bytes per weight and lower throughput. The expected monotonic ordering would be Q2_K > Q3_K_M > Q4_K_M > Q6_K > Q8_0. Our measured ordering is Q2_K > Q4_K_M > Q8_0 $\approx$ Q3_K_M > Q6_K.

We attribute the anomaly primarily to **kernel implementation overhead** in the llama.cpp Q6_K de-quantization path. The Q6_K format packs 6-bit weights using a combination of a 4-bit lookup table and 2-bit residuals, requiring more complex de-packing logic per block compared to Q8_0 (single 8-bit lookup) or Q4_K_M (well-optimized 4-bit NEON kernel with vectorized scale application). The de-quantization overhead partially offsets Q6_K's memory bandwidth advantage, yielding effective throughput below its bandwidth consumption would predict.

Similarly, Q3_K_M's moderate throughput (4.91 tokens/s despite 3-bit weights) reflects the overhead of its medium-complexity block structure, which requires scale factor lookups at finer granularity than Q4_K_M's implementation.

This finding highlights that the throughput ordering of quantization variants is not determined by model size alone, but by the joint optimization of memory bandwidth and kernel de-quantization efficiency. Practitioners should treat Q6_K with caution on ARM CPU backends.

B. Memory Bandwidth as the Decode Bottleneck

The context-length invariance result (Section IV-B) provides strong evidence that decode throughput on this device is **memory-bandwidth saturated** by weight loading. For Q4_K_M at 5.32 tokens/s, each decode step loads approximately 2.0 GB of weights, implying an effective DRAM bandwidth utilization of $2.0 \times 5.32 \approx 10.6$ GB/s for weight loading alone. The KV-cache at 2048-token context contributes at most 233 MB per step, less than 11% of weight traffic. This explains why even an 8$\times$ increase in context length produces less than 3% throughput variation.

C. Deployment Recommendations

Default recommendation: Q4_K_M. At 5.32 tokens/s, 2.0 GB, and 93.3% accuracy with low variance (std 0.52), Q4_K_M provides the best overall operating point for 6 GB devices. Though Q2_K is slightly faster and equally accurate, Q4_K_M's lower performance variance and broad community ecosystem support make it the pragmatic default.

Accuracy-critical deployments: Q3_K_M or Q8_0. Both variants achieve 100% on our factual evaluation suite. Q3_K_M (4.91 tokens/s, 1.6 GB) is the best choice for 4 GB devices and memory-constrained deployments. Q8_0 (4.95 tokens/s, 3.4 GB) is Pareto-optimal for 6 GB devices seeking perfect accuracy with competitive throughput.

Speed-critical deployments: Q2_K. At 5.66 tokens/s and only 1.3 GB, Q2_K is the fastest practical option. The 93.3% accuracy is acceptable for many latency-sensitive applications.

Avoid Q6_K on CPU backends. Q6_K is strictly dominated in both accuracy (86.7% vs. 93.3–100% for all others) and throughput (3.98 tokens/s, the slowest practical variant). Q4_K_M is faster, smaller, and more accurate; Q8_0 is more accurate and nearly 25% faster. This recommendation may not apply to GPU-accelerated backends where the Q6_K de-quantization overhead differs.

Use 4 threads on heterogeneous ARM. On the Tensor G2 (and similar big.LITTLE ARM SoCs), 4 threads matches the performance-core count and delivers peak throughput. Do not use all available threads.

Context length management. Decode throughput is context-invariant through 2048 tokens; applications should not artificially truncate context for performance reasons. TTFT grows linearly with context, so responsiveness decisions should be driven by TTFT budgets, not throughput concerns.

VI. Limitations and Future Work

A. Current Limitations

Single-device evaluation. All results are from a single Google Pixel 6a. The Tensor G2's specific characteristics may not generalize to Snapdragon, MediaTek, or Apple ARM SoC families.

Factual accuracy scope. The 15-question evaluation covers basic factual recall; more rigorous evaluation (MMLU, GSM8K, HumanEval) would provide broader quality characterization. The surprising non-monotonic accuracy ordering warrants further investigation.

Working memory measurement only. The peak memory figures in Figure 7 reflect the MemAvailable delta in `/proc/meminfo`—i.e., the KV-cache plus activation working memory—not the full RSS including memory-mapped

model weights. The model weights themselves contribute an additional 1.3–3.4 GB depending on variant, loaded via mmap.

Battery measurement granularity. The main benchmark sweep was conducted over WiFi ADB with the device unplugged. The session drained the battery from 78% to approximately 2% (∼76% total), corresponding to a rough session-level estimate of ∼2.1%/1k tokens across ∼35,000 output tokens. However, Android's `dumpsys battery` API reports integer-percentage values only; each individual 30-second trial consumes far less than 1% of charge, so per-trial battery tracking is noise-dominated (63 of 277 trials show a 1% drop, the rest 0%). Per-variant energy comparison therefore cannot be reliably extracted from this dataset. Accurate per-variant measurements require either a hardware power monitor (e.g., Monsoon Solutions) or longer-duration trials (≥256 output tokens) to amplify the per-trial signal above the 1% threshold.

Thermal trajectory not characterized. Despite a 120-second cooldown between experiments, cumulative thermal state over the 3-hour benchmark session may have affected later results. We report results in run order but do not model thermal drift explicitly.

B. Future Work

WikiText-2 perplexity evaluation. While on-device factual accuracy provides a practical measure of downstream quality, perplexity on a held-out corpus (WikiText-2 or C4) would enable comparison against published quantization papers and validate our accuracy-ordering results.

Cross-platform comparison. Repeating this benchmark on Snapdragon 8 Gen 3, Apple A-series, and MediaTek Dimensity platforms would establish whether the non-monotonic throughput finding is universal across ARM GGUF inference or specific to the Tensor G2's memory subsystem.

GPU and NPU acceleration. The Tensor G2 includes an Arm Mali-G710 GPU and a Google TPU block. Integrating these via OpenCL or delegating to TFLite's NNAPI backend could dramatically change the efficiency–quality Pareto frontier.

Context lengths beyond 2048. The context-invariance finding should be tested at 4096 and 8192 tokens, where KV-cache growth represents a larger fraction of per-step memory traffic and may begin to affect throughput.

Alternative frameworks. Evaluating MLC-LLM [7] and a future stable ExecuTorch [11] release on the same hardware would clarify whether the Q6_K anomaly is llama.cpp-specific or intrinsic to ARM NEON.

VII. Conclusion

We have presented a rigorous, multi-factor benchmark of six GGUF quantization variants of Llama 3.2 3B Instruct on a Google Pixel 6a (Tensor G2, 6 GB LPDDR5), yielding 277 measurements across four context lengths (256–2048 tokens), five quantization levels, and a thread-count scaling study.

Our primary empirical findings are:

1) **Non-monotonic throughput**: Q6_K (6-bit, 2.7 GB) delivers 3.98 tokens/s—slower than Q4_K_M (5.32 tokens/s) and Q8_0 (4.95 tokens/s) due to kernel dequantization overhead on ARM NEON.
2) **Extended context invariance**: Decode throughput varies by less than 3% from 256 to 2048 tokens, confirming mobile CPU inference is memory-bandwidth saturated by weight loading at all practical context lengths.
3) **Thread-count sweet spot**: 4 threads is optimal on the Tensor G2's heterogeneous cluster; 8 threads degrades throughput by 20% due to little-core load imbalance.
4) **Accuracy non-monotonicity**: On-device factual accuracy does not monotonically improve with bit-width across all five variants (86.7–100%): Q3_K_M (3-bit) and Q8_0 (8-bit) achieve perfect 100%, Q2_K and Q4_K_M score 93.3%, and Q6_K scores lowest at 86.7% despite having more bits per weight than Q4_K_M. Q6_K is strictly dominated on both accuracy and throughput, confirming it as the weakest deployment choice.
5) **Q4_K_M recommended; Q8_0 Pareto-optimal on accuracy**: The Pareto frontier is defined by Q8_0 (100%, 4.95 tokens/s) and Q2_K (93.3%, 5.66 tokens/s). Q4_K_M (2.0 GB, 5.32 tokens/s, 93.3%) offers the best overall balance for 6 GB deployment due to its lower variance and broad community ecosystem support.

These findings provide actionable guidance: use Q4_K_M as the default, Q3_K_M for accuracy-critical applications, Q2_K for latency-constrained deployments, 4 threads on big.LITTLE ARM, and avoid Q6_K on CPU backends. As on-device AI continues to proliferate, hardware-specific benchmarks of this kind are essential for evidence-based quantization selection.

References

[1] Meta AI, "Llama 3.2: Lightweight and Vision Language Models," Meta AI Blog, September 2024. [Online]. Available: https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/

[2] G. Gerganov et al., "llama.cpp: Efficient LLM inference in C/C++," GitHub Repository, 2023–2025. [Online]. Available: https://github.com/ggerganov/llama.cpp

[3] G. Gerganov, "GGUF: A File Format for Language Model Weights," GitHub, llama.cpp project, 2023. [Online]. Available: https://github.com/ggerganov/ggml/blob/master/docs/gguf.md

[4] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," in *Proc. ICLR*, 2023.

[5] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," in *Proc. MLSys*, 2024.

[6] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," in *Proc. ICML*, 2023.

[7] T. Chen et al., "MLC-LLM: Universal LLM Deployment Engine with ML Compilation," arXiv preprint arXiv:2404.01373, 2024.

[8] Qualcomm Technologies, Inc., "On-Device AI: Large Language Models on Snapdragon," Qualcomm AI Research Technical Report, 2024. [Online]. Available: https://www.qualcomm.com/research/artificial-intelligence/ai-research

[9] R. Yi et al., "EdgeMoE: Fast On-Device Inference of MoE-Based Large Language Models," arXiv preprint arXiv:2308.14352, 2023.

[10] Y. Song et al., "PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU," in *Proc. SOSP*, 2024.
[11] Meta AI, "ExecuTorch: On-Device AI across Mobile, Embedded, and Edge for PyTorch," GitHub Repository, 2024. [Online]. Available: https://github.com/pytorch/executorch
[12] AI@Meta, "Llama 3 Model Card," GitHub, 2024. [Online]. Available: https://github.com/meta-llama/llama3
[13] G. Gerganov, "GGML: Tensor Library for Machine Learning," GitHub Repository, 2023–2025. [Online]. Available: https://github.com/ggerganov/ggml

APPENDIX

A. Build Configuration

The llama.cpp library was compiled from commit `b1-1a29907` using the Android NDK version 29.0.14206865. Compilation flags:

```
-O3 -march=armv8.2-a+dotprod+fp16
-DGGML_USE_CPU_HBM=OFF
-DLLAMA_BUILD_TESTS=OFF
```

The resulting `llama-cli` binary was pushed to `/data/local/tmp/` on the device and executed via ADB shell. The Android app was compiled with Android NDK 29.0.14206865, targeting arm64-v8a, using CMake 4.2.3 and Gradle 8.x. All source code, benchmark scripts, and raw results are available in the project repository.

B. Invocation Template

Each trial was executed with the following command template:

```
./llama-cli \
  -m <model_path>.gguf \
  -p "<system_prompt><user_prompt>" \
  -n 128 \
  --ctx-size <context_length> \
  --seed <trial_seed> \
  -t <thread_count> \
  --log-format json
```

C. Statistical Methodology

Means and standard deviations in Table I are computed across 15 samples per variant per context (5 trials $\times$ 3 prompt types). Warmup runs (2 per configuration) are excluded. Outlier rejection was not applied; all completed trials within the timeout threshold are included in analysis.

The 15-question factual evaluation suite covers four categories:

Arithmetic (5): 17×8, $\sqrt{144}$, 2^{10}, 15% of 200, prime factor product $2 \times 3 \times 7$.

Geography (5): Capital of Japan (Tokyo), largest country by area (Russia), capital of Australia (Canberra), longest river (Nile), number of continents (7).

History/Science (5): Year of first iPhone (2007), chemical symbol for gold (Au), number of planets (8), planet closest to Sun (Mercury), year WWII ended (1945).

Each prompt instructs the model to respond with only the answer. Scoring uses case-insensitive substring matching.